

Universidade Tecnológica Federal do Paraná

Engenharia de Computação

Relatório Final Banco de Dados 2

Aluno: Guilherme Rodrigues dos Santos
Aluno: Luiz Eduardo Caldas Kramer
Aluno: Matheus Deodato Cadamuro
Aluno: Nailson Francisco da Silva Chagas
Professor: Ives Venturi Pola

Junho
2025

Conteúdo

1	Introdução	1
2	Diagrama Entidade-Relacionamento	1
3	Geração de Dados	2
3.1	Ferramentas Utilizadas	2
3.2	Quantitativo de Dados Gerados	3
3.3	Lógica de Geração	3
3.4	Exportação do Script SQL	3
4	Triggers	3
4.1	Trigger 1: Atualização de Estoque	3
4.2	Trigger 2: Cálculo de Preço e Atualização de Transação	4
4.3	Trigger 3: Remoção de Produto de Promoções Ativas	4
5	Análise de Índices e Desempenho de Consultas	4
6	Funções	9
6.1	Função 1: Consulta de Estoque de Produto	9
6.2	Função 2: Relatório de Vendas por Período	9
6.3	Função 3: Promoções Ativas por Produto	9
7	Views	9
7.1	View 1: Vendas por Período e Cliente	9
7.2	View 2: Produtos Mais Vendidos	10
7.3	View 3: Desempenho de Promoções	10
8	Auditoria de Clientes	10
8.1	Tabela de Auditoria	10
8.2	Triggers de Auditoria	10
8.2.1	Insert: <code>tr_audit_clientes_insert</code>	10
8.2.2	Update: <code>tr_audit_clientes_update</code>	11
8.2.3	Delete: <code>tr_audit_clientes_delete</code>	11
8.3	Importância da Auditoria	11
9	Conclusão	11

1 Introdução

Esta Atividade Prática Supervisionada (APS), desenvolvida para a disciplina de Banco de Dados 2 da Universidade Tecnológica Federal do Paraná, apresenta a modelagem e implementação completa de um sistema comercial utilizando PostgreSQL. O projeto atende rigorosamente aos nove requisitos estabelecidos:

1. **Aplicação com dados correlacionados:** Foi desenvolvido um sistema de gestão comercial que integra entidades inter-relacionadas como clientes, fornecedores, produtos, transações, estoque e promoções, simulando operações reais de uma empresa varejista.
2. **Diagrama Entidade-Relacionamento:** Elaborado com a ferramenta DBeaver, representa visualmente as entidades, atributos, relacionamentos e cardinalidades do sistema.
3. **Modelo relacional e integridade:** Mapeamento para tabelas normalizadas com restrições de chaves primárias, estrangeiras, CHECK e UNIQUE implementadas via DDL.
4. **Script de criação:** Geração de scripts SQL para criação de esquema e tabelas com todas as restrições de integridade.
5. **População de dados:** Desenvolvimento de algoritmo em Python com biblioteca Faker para geração de dados sintéticos em larga escala (20+ mil registros).
6. **Índices e desempenho:** Criação estratégica de índices B-tree, GIN e compostos com análise comparativa de desempenho usando EXPLAIN ANALYZE.
7. **Funções:** Implementação de 3 funções para consultas recorrentes e operações de negócio diretamente no banco.
8. **Visões e auditoria:** Criação de 3 views analíticas e sistema completo de auditoria para a tabela de clientes.
9. **Triggers:** Desenvolvimento de 3 triggers para atributos derivados, controle transacional e registro de auditoria.

O relatório documenta cada etapa do ciclo de vida do banco de dados, desde a modelagem conceitual até técnicas avançadas de otimização e segurança, demonstrando na prática os princípios de sistemas gerenciadores de bancos de dados estudados na disciplina.

2 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento (DER) apresentado na Figura 1 foi gerado utilizando a ferramenta DBeaver, que oferece uma interface gráfica intuitiva para visualização e modelagem de bancos de dados.

O DER reflete a estrutura lógica do sistema, evidenciando as principais entidades da aplicação comercial, como Clientes, Fornecedores, Produtos, Transações, Itens de Transação, Promoções e Estoque. As cardinalidades e os relacionamentos entre essas

entidades foram cuidadosamente definidos para representar de forma fiel os processos de compra, venda e controle de estoque.

Este diagrama também permitiu validar a normalização do banco, garantindo que os dados estão organizados de forma eficiente e sem redundâncias desnecessárias. Além disso, facilitou a implementação das chaves estrangeiras e das restrições de integridade nas etapas posteriores do desenvolvimento.

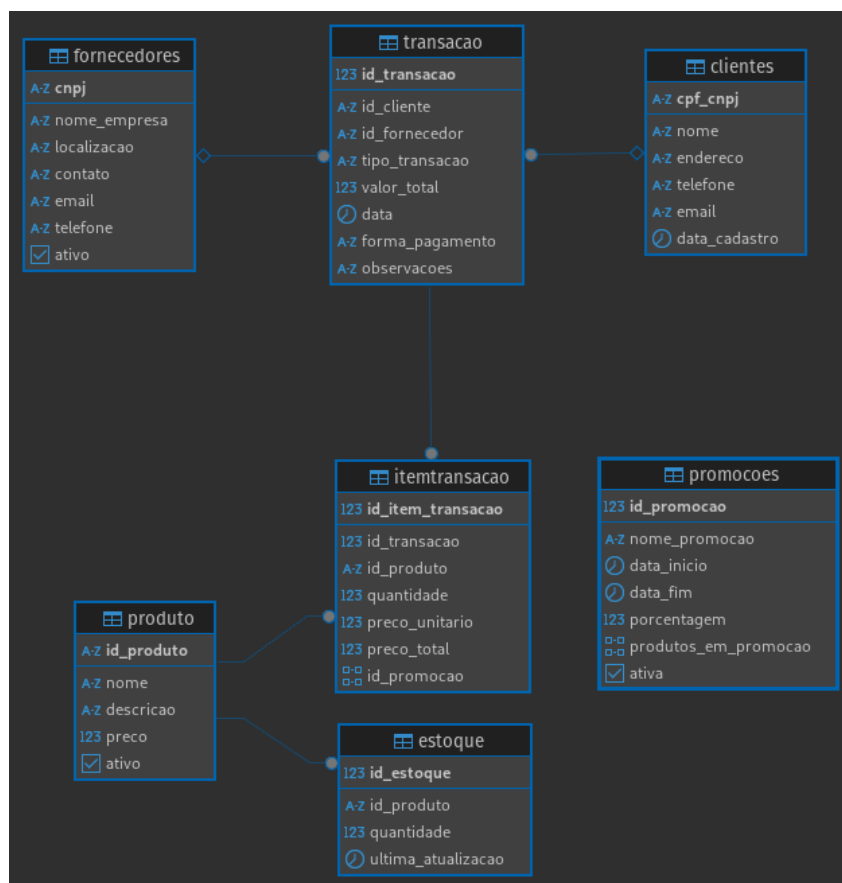


Figura 1: Diagrama Entidade-Relacionamento (DER) gerado no DBeaver

O DBeaver foi utilizado tanto para a criação das tabelas quanto para a geração do DER automaticamente a partir do modelo físico já implementado, acelerando a análise e validação do banco de dados.

3 Geração de Dados

Para realizar testes funcionais e validar a integridade das consultas SQL, foi necessário popular o banco de dados com uma quantidade significativa de dados fictícios. Para isso, foi desenvolvido um script em Python utilizando a biblioteca **Faker**, amplamente usada para geração de dados sintéticos realistas.

3.1 Ferramentas Utilizadas

A biblioteca **Faker** foi configurada com o local **pt_BR** para garantir a coerência com o formato brasileiro de nomes, endereços, telefones, CPF e CNPJ. Além disso, foi utilizada a biblioteca **random** para geração de números aleatórios e a classe **timedelta** da biblioteca **datetime** para manipulação de datas.

3.2 Quantitativo de Dados Gerados

O script foi configurado para gerar os seguintes volumes de dados:

- **Clientes:** 2.000 registros únicos, com dados como nome, CPF, endereço, telefone e e-mail.
- **Fornecedores:** 2.000 registros com CNPJ, nome da empresa, localização, pessoa de contato, telefone e e-mail.
- **Produtos:** 2.000 produtos com nome, descrição e preço.
- **Estoque:** Quantidade inicial de cada produto (entre 10 e 100 unidades).
- **Promoções:** 2.000 promoções associadas a conjuntos aleatórios de produtos, com nome, data de início e fim, porcentagem de desconto e status de ativação.
- **Transações:** 20.000 transações entre vendas e compras, com geração automática dos itens correspondentes, clientes ou fornecedores, forma de pagamento e observações.
- **Itens de Transação:** Cada transação contém entre 1 e 4 itens, considerando o controle de estoque e promoções aplicáveis.

3.3 Lógica de Geração

- **Unicidade de CPF e CNPJ:** Foram utilizadas funções para garantir que não houvesse duplicação de documentos, armazenando os valores já gerados em conjuntos Python.
- **Promoções e Estoque:** O script controla o estoque atual dos produtos, impedindo que transações de venda utilizem quantidades indisponíveis. Em transações de compra, o estoque é automaticamente incrementado. A aplicação de promoções nos itens de transação é feita aleatoriamente, podendo haver mais de uma promoção por item.
- **Data e Forma de Pagamento:** As datas das transações e promoções são distribuídas aleatoriamente dentro de uma janela recente (últimos 60 dias). As formas de pagamento são escolhidas entre: dinheiro, cartão, boleto ou PIX.

3.4 Exportação do Script SQL

Ao final da execução, o script Python exporta todas as instruções `INSERT` geradas para um arquivo chamado `4_populate_tables.sql`, permitindo a carga direta no banco de dados PostgreSQL.

4 Triggers

Triggers (ou gatilhos) são mecanismos que permitem executar procedimentos automáticos no banco de dados em resposta a eventos como inserções, atualizações ou deleções. A seguir, são descritas as triggers desenvolvidas para garantir a integridade e a consistência dos dados no sistema.

4.1 Trigger 1: Atualização de Estoque

A trigger `tr_atualizar_estoque` é acionada automaticamente antes da inserção de um novo item em uma transação. Sua função é:

- Verificar o tipo da transação (`VENDA` ou `COMPRA`).
- Atualizar a quantidade em estoque conforme o tipo:
 - Se `VENDA`, subtrai a quantidade do estoque (impedindo caso não haja estoque suficiente).
 - Se `COMPRA`, soma a quantidade ao estoque existente.
- Atualizar o campo `ultima_atualizacao` da tabela `Estoque`.

Essa trigger é fundamental para garantir que os dados de estoque estejam sempre consistentes com as operações registradas.

4.2 Trigger 2: Cálculo de Preço e Atualização de Transação

A trigger `tr_calcular_preco_e_atualizar_transacao` é ativada após a inserção de um item de transação. Ela é responsável por:

- Calcular o preço unitário do produto com base em promoções ativas, caso existam.
- Evitar que o preço final fique negativo ou nulo.
- Atualizar os campos `preco_unitario` e `preco_total` na tabela `ItemTransacao`.
- Atualizar o valor total da transação na tabela `Transacao`.

Essa trigger centraliza a lógica de cálculo de preços e descontos diretamente no banco, eliminando dependências no código da aplicação.

4.3 Trigger 3: Remoção de Produto de Promoções Ativas

A trigger `tr_verificar_promocoao_ao_atualizar_produto` é ativada após a atualização de um produto e tem como objetivo:

- Verificar se o produto foi desativado.
- Caso positivo, removê-lo automaticamente das promoções ativas em que esteja cadastrado.
- Registrar uma notificação com `RAISE NOTICE` indicando a ação.

Essa trigger impede que promoções ativas contenham produtos que já não estão mais disponíveis para venda, preservando a integridade das campanhas promocionais.

5 Análise de Índices e Desempenho de Consultas

Nesta seção, analisamos o impacto da criação de índices sobre o desempenho de consultas em um banco de dados PostgreSQL. As medições foram realizadas utilizando o comando `EXPLAIN ANALYZE`, tanto antes quanto após a criação dos índices. A seguir, são apresentadas as consultas realizadas, seus planos de execução e uma análise comparativa dos resultados.

Consulta 1 — Nome de cliente

Consulta:

```
SELECT * FROM Clientes WHERE nome LIKE 'Guilherme Rodrigues';
```

Sem índice:

```
Seq Scan on clientes  
Execution Time: 0.965 ms
```

Com índice `idx_clientes_nome`:

```
Index Scan using idx_clientes_nome on clientes  
Execution Time: 0.099 ms
```

Análise: A consulta foi acelerada com o uso do índice, embora a operação `LIKE` ainda exija verificação de filtro.

Consulta 2 — E-mail de cliente

Consulta:

```
SELECT * FROM Clientes WHERE email = 'maria46@example.net';
```

Sem índice:

```
Seq Scan on clientes  
Execution Time: 0.239 ms
```

Com índice idx_clientes_email:

```
Index Scan using idx_clientes_email on clientes  
Execution Time: 0.044 ms
```

Análise: O índice reduziu o tempo de execução, resultando em melhoria no desempenho..

Consulta 3 — Nome da empresa fornecedora

Consulta:

```
SELECT * FROM Fornecedores WHERE nome_empresa LIKE 'Guerra S/A';
```

Sem índice:

```
Seq Scan on fornecedores  
Execution Time: 0.979 ms
```

Com índice idx_fornecedores_nome_empresa:

```
Index Scan using idx_fornecedores_nome_empresa on fornecedores  
Execution Time: 0.096 ms
```

Análise: O uso do índice resultou em melhoria significativa no desempenho.

Consulta 4 — Nome de produto

Consulta:

```
SELECT * FROM Produto WHERE nome LIKE 'Tempora';
```

Sem índice:

```
Seq Scan on produto  
Execution Time: 0.646 ms
```

Com índice idx_produto_nome:

```
Bitmap Index Scan using idx_produto_nome  
Execution Time: 0.130 ms
```

Análise: O índice permitiu uso de Bitmap Index Scan, resultando em ganho expressivo.

Consulta 5 — Estoque por id_produto

Consulta:

```
SELECT * FROM Estoque WHERE id_produto = '789100000001';
```

Sem índice:

```
Seq Scan on estoque  
Execution Time: 6.415 ms
```

Com índice idx_estoque_id_produto:

```
Index Scan using idx_estoque_id_produto on estoque  
Execution Time: 0.057 ms
```

Análise: Grande redução no tempo de execução com uso do índice.

Consulta 6 — Promoções ativas por data

Consulta:

```
SELECT * FROM Promocoes  
WHERE ativa = TRUE AND data_inicio <= NOW() AND data_fim >= NOW();
```

Sem índice:

```
Seq Scan on promocoes  
Execution Time: 1.371 ms
```

Com índice idx_promocoes_perodo_ativas:

```
Bitmap Index Scan using idx_promocoes_perodo_ativas  
Execution Time: 0.262 ms
```

Análise: O uso do índice condicional reduziu significativamente o tempo.

Consulta 7 — Produto em promoção (GIN)

Consulta:

```
SELECT * FROM Promocoes  
WHERE ativa = TRUE AND produtos_em_promocao @> ARRAY['789100000001']::varchar[];
```

Sem índice:

```
Seq Scan on promocoes  
Execution Time: 1.251 ms
```

Com índice idx_promocoes_produtos_ativos_gin:

```
Bitmap Index Scan using idx_promocoes_produtos_ativos_gin  
Execution Time: 0.063 ms
```

Análise: Houve melhora com a aplicação do GIN.

Consulta 8 — Transações por id_cliente

Consulta:

```
SELECT * FROM Transacao WHERE id_cliente = '74930285160';
```

Sem índice:

```
Seq Scan on transacao  
Execution Time: 12.005 ms
```

Com índice idx_transacao_cliente_tipo_data:

```
Bitmap Index Scan on transacao  
Execution Time: 0.067 ms
```

Análise: Queda expressiva no tempo com uso de índice composto.

Consulta 9 — Transações por id_fornecedor

Consulta:

```
SELECT * FROM Transacao WHERE id_fornecedor = '89067423000132';
```

Sem índice:

```
Seq Scan on transacao  
Execution Time: 3.754 ms
```

Com índice idx_transacao_fornecedor_tipo_data:

```
Bitmap Index Scan on transacao  
Execution Time: 0.104 ms
```

Análise: Desempenho significativamente melhor com o índice adequado.

Consulta 10 — Transações por período

Consulta:

```
SELECT * FROM Transacao  
WHERE data >= '2025-07-01' AND data < '2025-08-01' ORDER BY data;
```

Sem índice:

```
Seq Scan + Sort on transacao  
Execution Time: 11.957 ms
```

Com índice idx_transacao_data:

```
Index Scan on transacao  
Execution Time: 4.544 ms
```

Análise: Houve melhoria, especialmente por evitar ordenação.

Consulta 11 — Transações por fornecedor, tipo e data

Consulta:

```
SELECT * FROM Transacao  
WHERE id_fornecedor = '89067423000132'  
AND tipo_transacao = 'COMPRA'  
AND data BETWEEN '2025-01-01' AND '2025-08-30';
```

Sem índice:

```
Seq Scan on transacao  
Execution Time: 7.727 ms
```

Com índice idx_transacao_fornecedor_tipo_data:

```
Bitmap Index Scan  
Execution Time: 0.056 ms
```

Análise: O índice composto otimizou substancialmente a consulta.

Consulta 12 — Itens por id_transacao

Consulta:

```
SELECT * FROM ItemTransacao WHERE id_transacao = 1000;
```

Sem índice:

```
Seq Scan on itemtransacao  
Execution Time: 13.918 ms
```

Com índice idx_itemtransacao_id_transacao:

```
Index Scan on itemtransacao  
Execution Time: 0.090 ms
```

Análise: Redução drástica de tempo com o índice simples.

Consulta 13 — Itens por id_produto

Consulta:

```
SELECT * FROM ItemTransacao WHERE id_produto = '789100000001';
```

Sem índice:

```
Seq Scan on itemtransacao  
Execution Time: 6.048 ms
```

Com índice idx_itemtransacao_produto_quantidade:

```
Bitmap Index Scan on itemtransacao  
Execution Time: 0.103 ms
```

Análise: Uso combinado de colunas no índice trouxe ganho considerável.

Consulta 14 — Itens com quantidade superior a 3

Consulta:

```
SELECT * FROM ItemTransacao  
WHERE id_produto = '789100000001' AND quantidade > 3;
```

Sem índice:

```
Seq Scan on itemtransacao  
Execution Time: 6.037 ms
```

Com índice idx_itemtransacao_produto_quantidade:

```
Bitmap Index Scan on itemtransacao  
Execution Time: 0.046 ms
```

Análise: O índice composto favoreceu a filtragem composta com ótimo desempenho.

Resultado

Os testes demonstram que o uso correto de índices pode reduzir significativamente o custo das consultas. Além disso, foi evidenciado que buscas com padrões como `ILIKE` ou uso de `ANY()` requerem índices especializados como GIN com `pg_trgm` para serem efetivos. A estratégia de criação e remoção dos índices permitiu quantificar o impacto real no plano de execução e justificar tecnicamente sua adoção.

6 Funções

A seguir, são descritas as principais funções criadas:

6.1 Função 1: Consulta de Estoque de Produto

A função `consultar_estoque_produto(p_id_produto)` retorna as informações de disponibilidade de um determinado produto, com base na sua chave primária.

- Exibe o nome do produto, quantidade atual em estoque, preço e o status.
- O status pode assumir os valores: `DISPONÍVEL`, `ESGOTADO` ou `INATIVO`, dependendo da quantidade e do estado de ativação do produto.
- Permite consultas rápidas para sistemas de venda e atendimento.

6.2 Função 2: Relatório de Vendas por Período

A função `relatorio_vendas_periodo(p_data_inicio, p_data_fim)` permite obter um resumo das vendas realizadas dentro de um intervalo de tempo específico.

- Para cada dia no período, retorna:
 - o total vendido,
 - a quantidade de itens vendidos,
 - o número de transações realizadas.
- Ideal para geração de relatórios gerenciais, dashboards e análise de desempenho diário.

6.3 Função 3: Promoções Ativas por Produto

A função `consultar_promocoes_ativas_produto(p_id_produto)` verifica se existem promoções ativas que incluem o produto especificado.

- Retorna as promoções vigentes no momento da consulta (com base em `CURRENT_TIMESTAMP`).
- Mostra o nome da promoção, o percentual de desconto e o intervalo de validade.
- Essencial para o sistema de vendas aplicar descontos automaticamente.

Essas funções fortalecem a lógica de negócio do sistema ao mover parte da inteligência diretamente para o SGBD, reduzindo a dependência de lógica replicada no backend da aplicação.

7 Views

As *views* (visões) foram criadas com o intuito de facilitar a análise dos dados armazenados no banco, oferecendo uma camada de abstração para consultas frequentes e complexas. Elas permitem que dados derivados sejam acessados como se fossem tabelas comuns, otimizando a interpretação das operações da empresa.

7.1 View 1: Vendas por Período e Cliente

A view `view_vendas_por_periodo_cliente` consolida as informações de vendas por cliente e período (mês e ano). Ela permite analisar o comportamento dos consumidores, o desempenho de vendas ao longo do tempo, além de facilitar a geração de relatórios por cliente.

- Agrupa as transações do tipo `VENDA` por cliente e por mês/ano.
- Mostra a quantidade de itens e produtos vendidos por transação.
- Permite observar a forma de pagamento mais recorrente.

7.2 View 2: Produtos Mais Vendidos

A view `view_produtos_mais_vendidos` reúne dados de desempenho de cada produto com base nas transações registradas, tanto de venda quanto de compra.

- Mostra a quantidade total vendida e comprada de cada produto.
- Informa o estoque atual e a última movimentação.
- Calcula o total vendido com base nos itens das transações.
- Estima o total de desconto aplicado em relação ao preço base.

Essa view é essencial para o controle de estoque, planejamento de compras e monitoramento das vendas de produtos com maior rotatividade.

7.3 View 3: Desempenho de Promoções

A view `view_desempenho_promocoas` foi criada para mensurar a eficácia das campanhas promocionais ativas.

- Mostra o número de transações e itens vendidos associados a cada promoção.
- Calcula o total de vendas gerado por cada promoção.
- Apresenta o valor total de descontos concedidos.
- Lista os produtos associados a cada campanha.

Com esta view, é possível verificar quais promoções realmente impactaram nas vendas e se a aplicação de descontos resultou em aumento significativo no volume comercializado.

8 Auditoria de Clientes

Para garantir a rastreabilidade e a segurança das informações, foi implementado um sistema de auditoria completo para a tabela `Clientes`. Toda alteração nos dados — seja inserção, atualização ou exclusão — é registrada de forma detalhada na tabela `AuditoriaClientes`.

8.1 Tabela de Auditoria

A tabela `AuditoriaClientes` armazena o histórico de modificações realizadas nos registros de clientes. Os principais campos incluem:

- **cpf.cnpj**: Identificador do cliente.
- **Valores antigos e novos**: Para nome, endereço, telefone, e-mail e data de cadastro.
- **Operação**: Tipo de alteração realizada — `INSERT`, `UPDATE` ou `DELETE`.
- **Usuário**: Nome do usuário que executou a operação, capturado via `current.user`.
- **Data/hora**: Momento exato da alteração.

8.2 Triggers de Auditoria

Foram implementadas três triggers, cada uma associada a um tipo específico de operação na tabela `Clientes`:

8.2.1 Insert: `tr_audit_clientes_insert`

Essa trigger é ativada após a inserção de um novo cliente. Ela armazena os dados incluídos e marca a operação como `INSERT`.

8.2.2 Update: `tr_audit_clientes_update`

Ativada após qualquer atualização em um registro de cliente. Essa trigger registra os valores antigos e os novos, permitindo uma comparação completa das modificações.

8.2.3 Delete: `tr_audit_clientes_delete`

Acionada após a exclusão de um cliente, essa trigger registra os dados que estavam presentes antes da remoção, com a operação marcada como `DELETE`.

8.3 Importância da Auditoria

A implementação dessa auditoria é essencial para garantir a integridade e confiabilidade do sistema. Permite:

- Investigar alterações indevidas ou erros em dados sensíveis.
- Criar trilhas de auditoria para fins legais ou administrativos.
- Oferecer mais transparência na gestão das informações dos clientes.

A auditoria foi implementada diretamente no nível do banco de dados, assegurando que qualquer alteração — independentemente da origem (aplicativo, script ou acesso manual) — seja registrada.

9 Conclusão

A realização desta Atividade Prática Supervisionada (APS) permitiu aplicar, de forma integrada, os principais conceitos estudados ao longo da disciplina de Banco de Dados 2. Por meio do desenvolvimento de um sistema comercial, envolvendo o planejamento e implementação de tabelas, diagrama Entidade-Relacionamento, geração e inserção de dados realistas, criação de índices otimizados, funções, visões, mecanismos de auditoria e triggers, assim foi possível compreender na prática a importância de cada etapa no ciclo de vida de um banco de dados relacional.

Além disso, os testes de desempenho e o uso de índices específicos, como os do tipo GIN para buscas textuais e arrays, evidenciaram na prática como decisões de projeto impactam diretamente na performance das consultas. A implementação de funções e views reforçou a importância de abstrair operações frequentes, facilitando consultas e relatórios gerenciais. Por fim, a auditoria de clientes demonstrou a relevância de manter registros confiáveis de alterações, garantindo maior segurança e rastreabilidade dos dados. Dessa forma, a APS cumpriu seu papel ao unir teoria e prática, consolidando o conhecimento adquirido.